

B6B36OMO — Objektový návrh a modelování, SW3 Smart Seafood Chain, 28. ledna 2026

URL: https://gitlab.fel.cvut.cz/B251_B6B36OMO/zubkovla/-/tree/SW3

Vysokoúrovňové požadavky (High-level requirements)

Podnikatelské požadavky (BRQ)

- BRQ1 — Zajistit **sledování původu mořských plodů v celém logistickém řetězci (LŘ)**.
- BRQ2 — Mít možnost **plné kontroly LŘ po stránce**:
 - BRQ2.1 — **ekonomické** (marže, náklady, efektivita, doby obrátů),
 - BRQ2.2 — **logistické** (přeprava, výkyvy poptávky, výroba),
 - BRQ2.3 — **technické** (stavy strojů, poruchy, údržba).
- BRQ3 — Mít možnost **auditovat provoz LŘ na základě reportů**.

Uživatelské požadavky (URQ)

Požadavky vznikají na základě identifikace potřeb a cílů. Při definování uživatelských požadavků se vštěpil pojem uživatelských příběhů (tzv. 'user story' — součást tzv. EPIC v agilní praxi). Uživatelský příběh vzniká v následující podobě: Jako *[role osoby]*, já *[chci]*, *[tak aby]*.

- URQ1 — Jako *Zákazník*, chci **nakoupit mořské plody v určeném stavu a kvalitě ve stanovené době na určeném místě za určitou cenu**, tak abych **uspokojil svou potřebu (poptávka)**.
- URQ2 — Jako *Rybář, Zpracovatel, Tržnice, Kuchyně, Distributor, Prodejce* chci **prodat** úlovek dál, tak abych **získal peněžní prostředky**.
- URQ3 — Jako *Přepravce* chci **předat** úlovek dál, tak abych **získal peněžní prostředky**.
- URQ4 — Jako *Rybář, Zpracovatel, Sklad, Tržnice, Kuchyně, Přepravce, Prodejce* chci **předat** úlovek dál, tak abych **uvolnil kapacitu**.
- URQ5 — Jako *Manažer logistického řetězce (MLŘ)*, chci **plnit business požadavky**, tak aby **logistický řetězec byl agilní a já získal roční prémii**.
- URQ6 — Jako *Inspektor*, chci **kontrolovat kvalitu výrobků, detekovat a předbíhat odchylkám v režimu LŘ, detekovat a předbíhat poruchám strojů**, tak aby **logistický řetězec byl neustále v provozu**.
- URQ7 — Jako *Opravář*, chci **rychlé upravovat nahlášené poruchy strojů**, tak aby **logistický řetězec byl neustále v provozu**.
- URQ8 — Jako *Kuchař/Pracovník*, chci **mít možnost neomezené pracovní aktivity**, tak **abych byl co nejlépe finančně ohodnocen**.

Funkční požadavky (FRQ)

V této fázi funkční požadavky jsou formulovány jako sady klíčových schopností systému. Tento přístup upřednostňuje srozumitelnost a kontext před striktní formalizací, která bude doplněna v následných krocích návrhu (v případech užití a detailních funkčních systémových požadavcích).

- **Blokčejn**

- **FRQ1 (definice)** — systém musí zabezpečit plnou dohledatelnost každé šarže mořských plodů od ulovení až po konečného zákazníka („od lodě do talíře“) prostřednictvím registrace jednoznačného ID šarže a sekvence neměnných blockčejnových transakcí.
- **FRQ2 (implementace)** — každá operace související s materiálovým tokem v logistickém řetězci (lov, přeprava, zpracování, skladování, vaření, balení, prodej) musí být zaznamenána jako blockčejnová transakce obsahující minimálně: strany, čas, parametry (teplota, kvalita, cena), podpis a hash předchozí transakce. Může být zaznamenán stav, název produktu a jeho složení.
- **FRQ3 (bezpečnost a integrita)** — systém musí automaticky detekovat a blokovat vícenásobný prodej stejného zboží (double-spending) a jakýkoliv pokus o zpětnou manipulaci s parametry transakcí (například zpětnou změnu zapsané teploty), včetně simulace celých falešných scénářů (blockchain tampering).
- **FRQ4 (uživatelský přístup)** — zákazník nebo autorizovaný zaměstnanec musí mít možnost naskenovat QR/NFC kód na produktu a zobrazit kompletní odpovídající historii šarže včetně teplotního režimu, aplikovaných procesů, případně změn stavů produktu a odpovědných subjektů a osob.

- **Provoz a logistika**

- **FRQ5 (ekonomika materiálového toku)** — systém musí evidovat a automaticky vypořádat finance při každém předání šarže a materiálů (cena, marže) a uvolňovat kapacitu skladů/výroby. Iniciativa k inicializaci toku může pocházet od zákazníka (*"tažení"*) či od výrobce (*"tlačení"*). V případě nedostatku peněz může být poskytnut obchodní. O jeho výši, době splátek rozhoduje MLŘ (viz. **FRQ11**).
- **FRQ6 (třídy produktů, kanály)** — systém musí podporovat více kanálů (např. čerstvé vs. mražené produkty) s vlastními pravidly a účastníky.
- **FRQ7 (dodržení režimu logistického řetězce, IoT a chlazení)** — systém musí umožňovat připojení IoT senzorů (teplota, vlhkost, otřesy) k přepravním a skladovacím jednotkám a automaticky zaznamenávat a vyhodnocovat dodržování chladicího řetězce v reálném čase.

- **FRQ8 (reakce na problémy)** — systém musí automaticky upozorňovat příslušné role při detekci anomálií (teplotní odchylka, porucha stroje, případně zpoždění přepravy, překročení kapacity, vloupání atd.). Systém může upozorňovat příslušné role přímo, nebo používat k tomu vyhrazenou linku (typ + priorita + FIFO).
- **Řízení a role**
 - **FRQ9 (definuje řídicí role)** — MLŘ a Inspektor musí mít možnost aktivně procházet řetězec po článcích, kontrolovat stav, opotřebení strojů, provádět optimalizace procesů, kontrolovat dodržení kvality výrobků, jejich akce se zapisují to logů.
 - **FRQ10 (sestavení logistického řetězce)** — systém musí umožňovat MLŘ sestavovat LŘ, to jest skládat články LŘ za sebe podle funkcionality, určovat skladové, výrobní kapacity, objemy a způsoby přepravy, stanovovat bod rozpojení v LŘ (bod ve kterém tažení materiálového toku se mění na tlačení — do tohoto bodu zasahuje aktuální poptávka zákazníka).
 - **FRQ11 (nastavení logistického řetězce, KPI)** — systém musí umožňovat MLŘ provádět nastavení jednotlivých článků řetězce, včetně dob obrátů zásob, přepravy a plateb, výše a doby obchodního úvěru, stanovovat KPI pro agilitu řetězce (např. čas od úlovku ke konečnému prodeji < 72 hodin, minimální ztráty < 2 % atd.), stanovovat odměny pracovníků a prémie; nakonec testovat průchod materiálového toku přes LŘ.
 - **FRQ12 (spotřeba pracovní síly a strojů)** — systém musí umožňovat kuchařům/pracovníkům evidovat odpracovaný čas, množství zpracovaných surovin (materiálů), množství zpracovaných/prodaných šarží pro automatický výpočet výkonnostní odměny. Systém musí evidovat spotřebu strojů a umožnit vyčíslit efektivitu logistického článku.
- **Simulace a reportování**
 - **FRQ13 (simulace)** — systém musí běžet jako diskretní simulace v jednom vlákně v časových taktech (např. 1 takt = 1 hodina), kdy se v každém taktu sekvenčně zpracovávají všechny události a operace (lov, přeprava, zpracování, vaření, prodej, poruchy, opravy atd.) a transakce všech entit.
 - **FRQ14 (poruchy)** — systém musí modelovat poruchy strojů a robotů na základě opotřebení, generovat události poruch (upozornění) a přidělovat omezený počet opravářů (priorita a FIFO).
 - **FRQ15 (reporty)** — systém musí poskytovat přehledy o LŘ v reálném čase v podobě generovaných reportů z pohledu 1) **materiálového toku** — původ, průchod; 2) **zúčastněných stran** — ziskovost, podíly v LŘ, zdržení, použité vybavení, doby obratu zásob (lead-time), vytížení (efektivita); 3) **spotřeby** — peněz a surovin; 4) **bezpečnosti** — pokusy o manipulaci, vícenásobný prodej, krádeže; 5) **transakce** — stav financí, zásob a transakcí za každý takt; 6) **výpadků** — jejich doby a zápisy o opravách.

- **Bonusové požadavky**

- **FRQ16 (dynamická poptávka)** — zákazníci mohou vysílat požadavky do kanálů a ostatní strany budou na ně reagovat podle dostupnosti a ceny. Dynamická poptávka může mít sezonní složku a změnu preferenci (chutí) zákazníků.
- **FRQ17 (stavový automat ryby)** — ryba musí procházet stavy (ulovena → rozpracována → připravena → prodána) s omezením jednoho přechodu mezi stavy za takt.
- **FRQ18 (rekonstrukce stavu)** — možnost zrekonstruovat přesný stav entity v libovolném minulém taktu na základě přehrání historie blockchainu (Technická realizace: Event Sourcing / Memento).
- **FRQ19 (report celého logistického řetězce)** — systém umožní stanovit finanční ukazatele logistického článku na základě jeho ziskovosti, kapitálových výdajů na stroje, aktuální spotřeby surovin v článku a vytížení pracovníků. Články se skládají do řetězců a tak lze vyčíslit ziskovost i další ukazatele celého LŘ.
- **FRQ20 (dynamická změna linek)** — umožnit přeskupení sekvence strojů na lince pro výrobu jiného jídla (např. sushi místo rybího salátu) dle zadaného receptu. Navazuje na dynamickou poptávku (**FRQ16**) nebo adresovat poruchy strojů z důvodů většího opotřebení (**FRQ14**).

Kvalitativní požadavky a omezení (NFRQ)

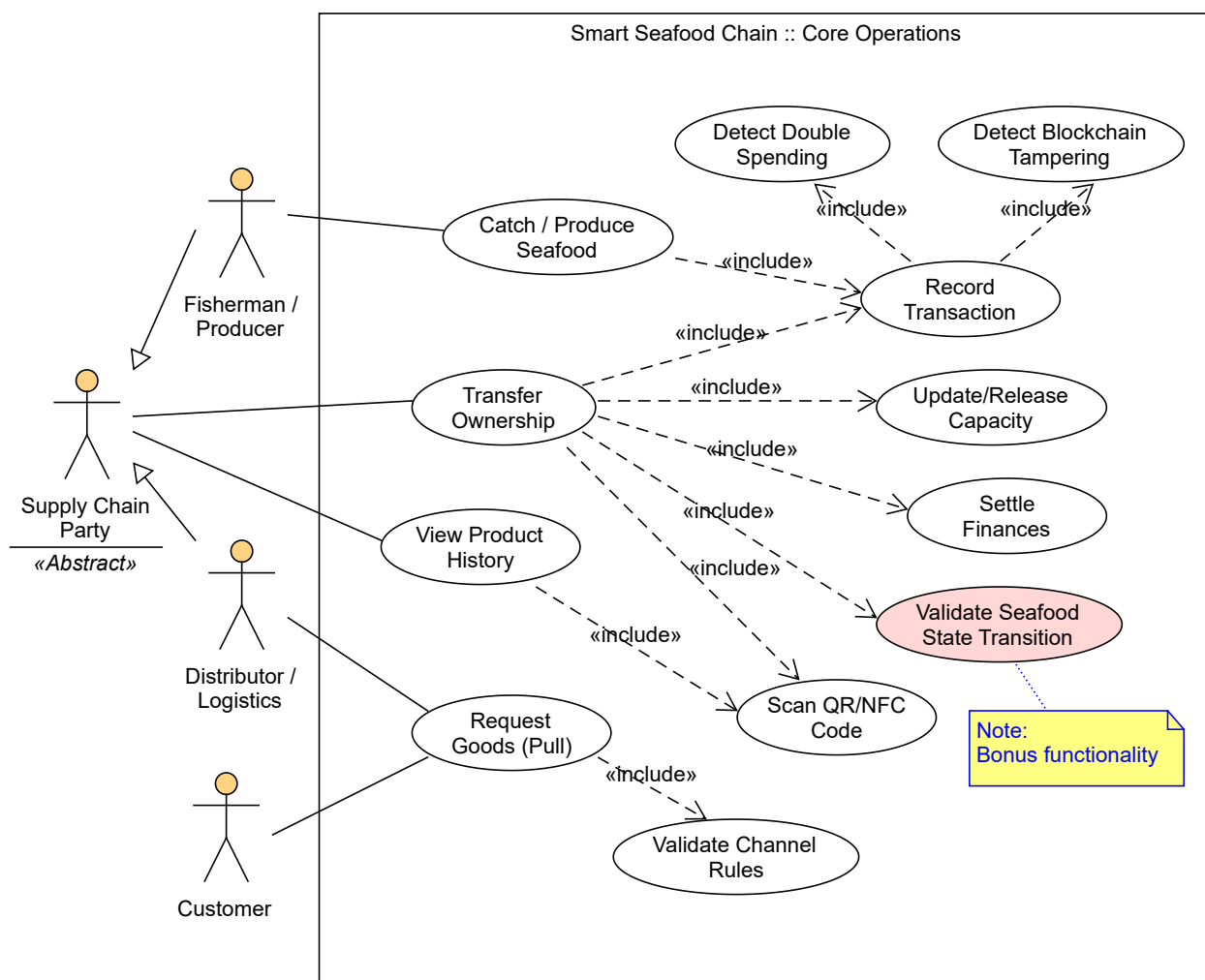
- **NFRQ1 (formát konfigurace)** — celý logistický systém (entity, linky, kanály, stroje, počáteční zásoby) musí být konfigurovatelný přes externí YAML soubor.

Diagram případů užití (Use Case Diagram)

Pomůže určit hranice IT systému, co bude systém umožňovat.

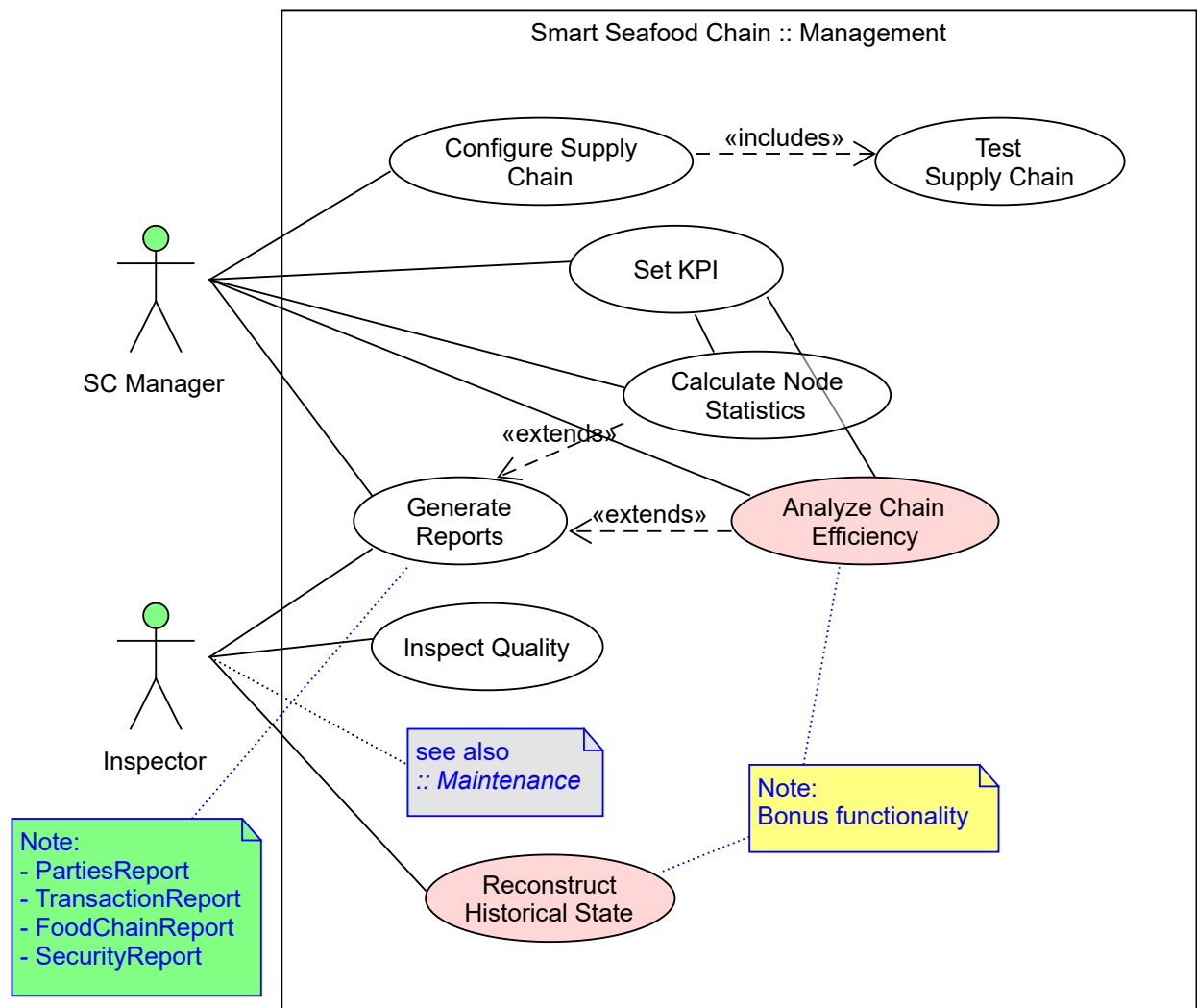
Základní operace

Následující diagram 'Core Operations' zachycuje klíčové procesy toku zboží a financí v celém řetězci. Využíváme zde abstraktního aktéra *Supply Chain Party*, který sjednocuje společné operace všech účastníků, jako je převod vlastnictví nebo náhled historie. Diagram rovněž vizualizuje integraci bezpečnostních kontrol přímo do transakčního procesu



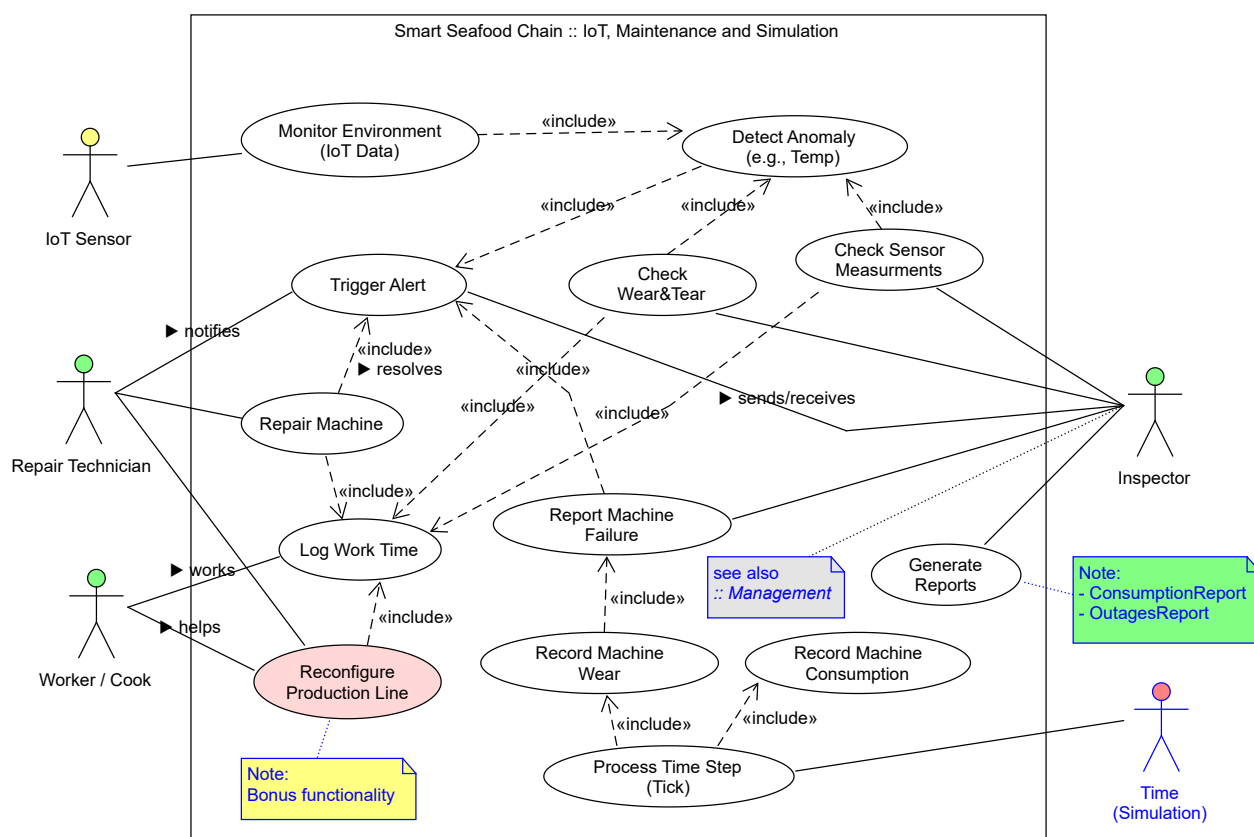
Nastavení řetězce

Diagram 'Management' odděluje strategické řízení od operativy. Role *SC Managera* se zaměřuje na konfiguraci řetězce a vyhodnocování KPI, zatímco role *Inspektora* pokrývá audit kvality a forenzní analýzu historie. Barevným odlišením jsme zvýraznili pokročilé analytické funkce definované jako bonusové požadavky.



Provoz a údržba — monitorování, upozornění, běh simulace

Diagram 'IoT, Maintenance and Simulation' modeluje technickou vrstvu systému, kde klíčový aktér *Time* řídí diskrétní simulaci. Schéma například detailně zobrazuje životní cyklus incidentu od automatické detekce opotřebení senzorem, přes vyvolání upozornění, až po fyzickou opravu technikem a následné uzavření incidentu. Stejný cyklus může vyvolat i *Inspektor*, který svou včasnou kontrolou může předejít havárii a dlouhodobé odstávce výrobní linky.



Detailní funkční systémové požadavky (SFR, low-level functional requirements)

Tyto požadavky detailně rozpracovávají vysokoúrovňové funkční požadavky (FRQ) a definují konkrétní chování systému v rámci jednotlivých Případů užití. V číslování zde navazujeme na odvozené v UC domény.

1. Jádru simulace a čas (Simulation)

Vychází z Use Case diagramu "IoT & Simulation", aktér Time.

- **SFR 1.1** — Systém řídí běh aplikace v diskrétních časových krocích (dále jen "takty").
- **SFR 1.2** — Systém umožní konfigurovat délku jednoho taktu v reálném čase (např. 1 takt = 1 hodina) prostřednictvím konfiguračního souboru.
- **SFR 1.3** — V každém taktu systém sekvenčně vyvolá metodu pro zpracování chování (`handleTick`) u všech aktivních entit (Party, Employee, Machine, Sensor).
- **SFR 1.4** — Systém ukončí simulaci po dosažení konfigurovaného maximálního počtu taktů nebo na příkaz uživatele.

2. Logistika a Blockchain (Core Operations)

Vychází z Use Case diagramu "Core Operations", aktér Supply Chain Party.

- **SFR 2.1 (Transfer)** — Systém provede změnu vlastníka šarže (Batch) z odesílatele na příjemce na základě platného požadavku.
- **SFR 2.2 (Blockchain)** — Při každé změně vlastníka nebo stavu šarže systém vytvoří nový neměnný záznam (Blok) v blockchainu.
- **SFR 2.3 (Security)** — Před zapsáním transakce systém ověří, zda daná šarže již nebyla prodána jinému subjektu (detekce Double Spending).
- **SFR 2.4 (Integrity)** — Systém při každém přístupu k historii šarže ověří validitu hashů v řetězci bloků (detekce Tampering).
- **SFR 2.5 (Finance)** — Systém automaticky převede virtuální peněžní prostředky z účtu kupujícího na účet prodávajícího ve výši dohodnuté ceny.
- **SFR 2.6 (Capacity)** — Systém ověří, zda má příjemce dostatečnou volnou kapacitu skladu před potvrzením transakce.
- **SFR 2.7 (Pull)** — Systém umožní odběrateli (Distributor, Customer) vytvořit poptávku (Request Goods), která se zařadí do seznamu úkolů pro dodavatele.

3. Provoz a údržba, Internet věcí (IoT & Maintenance)

Vychází z Use Case diagramu "IoT & Maintenance", aktéři Sensor, Technician, Worker.

- **SFR 3.1 (Senzory)** — Systém v každém taktu vygeneruje naměřené hodnoty (teplota, vlhkost) pro každý aktivní senzor připojený k šarži.
- **SFR 3.2 (Wear)** — Systém v každém taktu, kdy je stroj ve stavu `PROCESSING`, zvýší jeho hodnotu opotřebení (Wear Level) o definovanou konstantu.
- **SFR 3.3 (Failure)** — Pokud opotřebení stroje překročí kritickou mez (např. 90 %), systém s definovanou pravděpodobností změní stav stroje na `BROKEN`.
- **SFR 3.4 (Alerts)** — Systém vygeneruje událost typu `Alert`, jakmile dojde k poruše stroje nebo detekci anomálie v datech ze senzoru.
- **SFR 3.5 (Repair)** — Systém umožní přiřadit volného Technika (Repair Technician) k opravě stroje ve stavu `BROKEN`.
- **SFR 3.6 (Repair Time)** — Systém zablokuje Technika a stroj po dobu trvání opravy (konfigurovatelný počet taktů).
- **SFR 3.7 (Reconfig)** — Systém umožní změnit konfiguraci výrobní linky (přiřazení strojů pro jiný recept) pouze pokud je linka ve stavu `IDLE`.
- **SFR 3.8 (Work Log)** — Systém zaznamená každou hodinu práce zaměstnance (Worker, Cook, Technician) do jeho osobního výkazu pro výpočet mzdy.

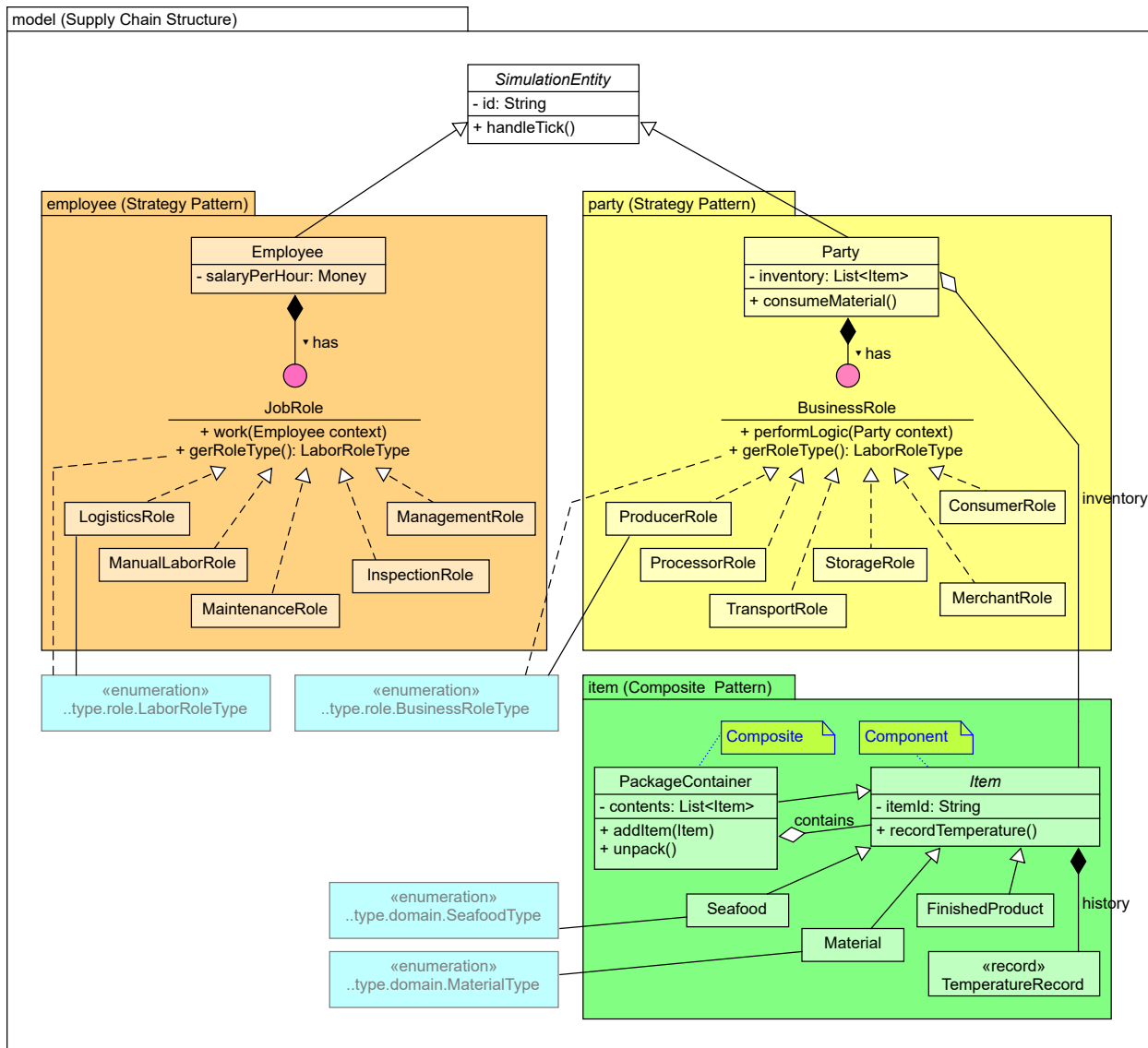
4. Řízení a Reporty (Management)

Vychází z Use Case diagramu "Management", aktéři SC Manager, Inspector.

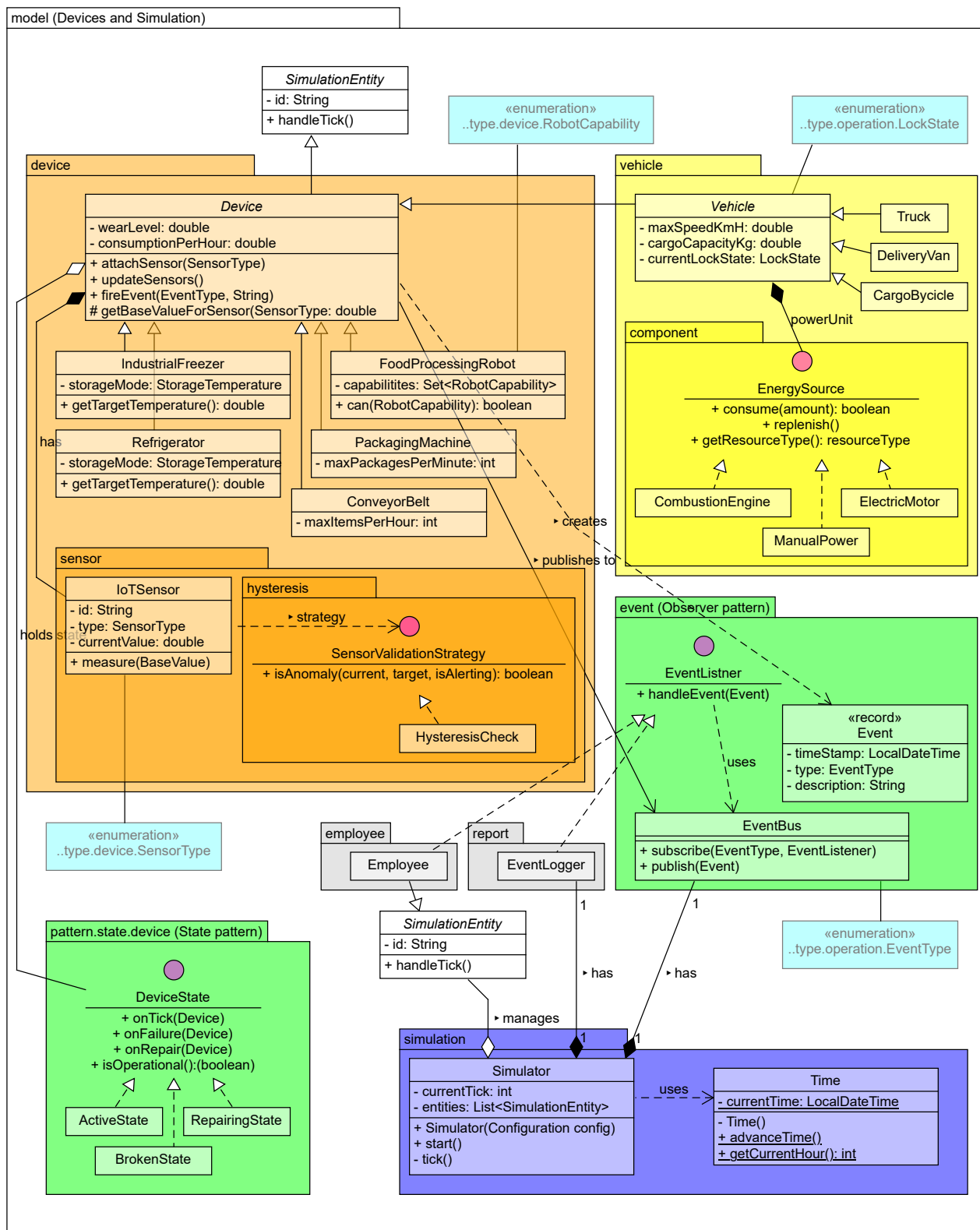
- **SFR 4.1 (Config)** — Systém načte počáteční konfiguraci řetězce (seznam účastníků, strojů, vztahů) z externího souboru formátu YAML při startu aplikace.
- **SFR 4.2 (KPI)** — Systém umožní Manažerovi definovat prahové hodnoty pro KPI (např. maximální doba dodání).
- **SFR 4.3 (Reporting)** — Systém na konci simulace vygeneruje textový soubor `FoodChainReport.txt` obsahující kompletní historii pohybu všech šarží.
- **SFR 4.4 (Reporting)** — Systém vygeneruje soubor `ConsumptionReport.txt` obsahující souhrn spotřeby energie a materiálů pro jednotlivé stroje. A další reporty.
- **SFR 4.5 (Audit – Bonus)** — Systém umožní Inspektorovi vyžádat rekonstrukci stavu konkrétní entity k libovolnému historickému taktu (využitím uložených událostí).
- **SFR 4.6 (Efficiency – Bonus)** — Systém vypočítá celkovou efektivitu řetězce jako poměr mezi celkovými náklady (provoz, odpisy) a celkovými tržbami z prodeje koncovým zákazníkům.

Diagram tříd (Class Diagram)

Základní model logistického řetězce (Supply Chain Structure)

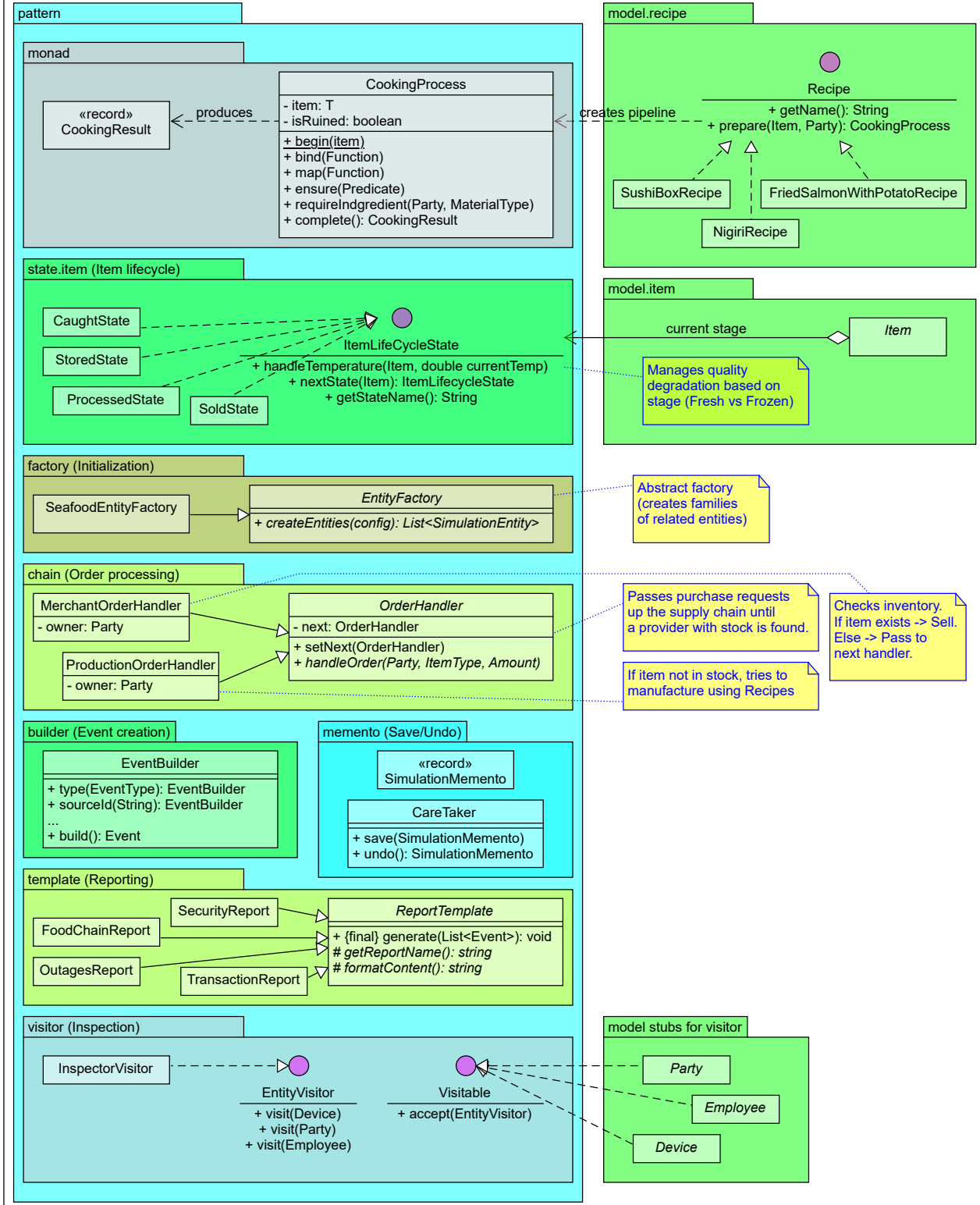


Zařízení, simulace a události (Devices & Simulation Engine)



Procesy, recepty, reporty (Process & Reporting)

seafood (Process and Logic)



Realizace

V rámci realizace byla implementována flexibilní strategie načítání dat, která podporuje jak načítání předdefinovaných konfigurací ve formátu `yaml` z classpath (adresář `resources` projektu nebo přímo `JAR` soubor), anebo ze souborů `yaml` dodaných uživatelem.

Deserializace dat se provádí prostřednictvím knihovny `Jackson` následovně:

- `ObjectMapper` používá instanci `YAMLFactory`, umožňující parsování YAML syntaxe.
- Po načtení obsahu souboru `ObjectMapper` mapuje jej na strukturu třídy `Configuration.java`.
- `Configuration.java` obsahuje statické třídy sloužící jako DTO a mají funkci kontejnerů pro nastavení jednotlivých tříd (`PartyConfig`, `DeviceConfig` atd.). Anotace `@Data` knihovny `Lombok` poskytují přístupové metody (getter a setter), které Jackson používá pro zápis hodnot do jednotlivých objektů.

Kroky načítání dat

`YamlConfigLoader` postupuje v těchto krocích:

1. **Resources:** aplikace hledá soubor v interních zdrojích (classpath/JAR).
2. **Filesystem:** Pokud není nalezen, hledá jej jako externí soubor na disku (lze používat bez rekompile).
3. **Fallback:** Pokud selže i to, automaticky se vygeneruje minimální nouzová konfigurace, aby aplikace nehavarovala.

Následně jednotlivé konfigurační soubory se použijí v Simulaci:

Inicializace a běh simulace

Centrální řídicí prvek aplikace je třída `SimulationController.java`, který orchestruje celou simulaci. Můžeme říct, že se jedná o jakousi *fasádu* systému, protože načítá všechny jeho prvky: abstraktní entitu `SimulationEntity`, od níž se odvozují všechny entity modelu (vkládají se do seznamu), soubor konfigurace popsány výše, a speciality: sběrnice událostí (`EventBus`), logování (`EventLogger`) a blockchain (`Blockchain`).

Simulace rovněž plní i funkci *controlleru* - používá globální třídu `Time` k běhu aplikace.

Poznámky: recepty obsahují názvy ryb. Je potřeba sledovat aby používané zpracovatelem recepty měly ty druhé ryb, které přísluší jejich mořské oblasti (revíru), jinak report zpracovatele `FactoryConsumptionReport` bude prázdný - mořské plody minou zpracovatele a půjdou v dodavatelském řetězci přímo do prodeje.

V kódu se hojně používají "chytré" výčtové typy s logikou. Ukázka sekvenčního diagramu je v adresáři projektu na gitlabu [Dokumentace/DiagramyUML/plantuml-output/SmartEnums_sequential.png](https://gitlab.com/Dokumentace/DiagramyUML/plantuml-output/SmartEnums_sequential.png)

Aplikace má předefinovaná v classpathu nastavení logistických řetězců, dle časové délky cyklu a prostorového rozsahu, od nejmenšího po největší:

scm-test.yaml (5 hodin), prague-sushi-luxury.yaml (3 dny), nordic-fresh.yaml (týden), baltic-frozen.yaml (dva týdny), russian-sea-dominator.yaml (měsíc).

Základní simulační jednotkou je 1 hodina, definována jako 60 minut.

Aplikace může být spuštěna s vlastními nastaveními příkazem: `java -jar target/smart-seafood-chain.jar edge-case.yaml`. Byl vytvořen konfigurační soubor `edge-case.yaml`, který testuje hraniční podmínky, jako je oblast s minimálním výlovem ("Dead Sea") či krachující rybář. Konfigurace také obsahuje záměrně poruchové mrazicí zařízení s extrémní spotřebou a neznámé druhy ryb ("GoldFish") pro ověření chování systému v nestandardních situacích.

Délka přehrávání simulace se nastavuje v parametru `tickDelayms` v souboru `simulationconfig`, výchozí nastavení je 1 vteřina, 0 - nejrychlejší varianta.

Reporty:

Po běhu systému se generují reporty:

Systém:

- **EventLog** - chronologický záznam úplně všech událostí v simulaci, který slouží jako „černá skříňka“ systému pro detailní zpětnou analýzu.
Pro vývojáře (ladění) a forenzní auditory. Jinými slovy, SecurityReport může hlásit, že systém je ok, ale v EventLogu bude vidět útok na systém a rollback.
- **StateReport** - zobrazuje aktuální stav zásob všech účastníků v reálném čase a potvrzuje celkovou validitu celého blockchainu. V podstatě se ukládá na konci, a proto ukazuje stav na účastníků na konci simulace. *Pro* skladové manažery a auditory blockchainu.

Reporting:

- **SecurityReport** - vyhodnocuje bezpečnostní incidenty a integritu dat po detekci útoku a následné obnově stavu světa (pattern Memento).
V podstatě vždy ukazuje Ok.
Pro bezpečnostní analytiky a správce systému.
- **FoodChainHistoryReport** - poskytuje kompletní historii životního cyklu produktu – od okamžiku ulovení ryby až po její prodej koncovému zákazníkovi. *Pro* zákazníky (ověření původu) a auditory (Traceability, Sledovatelnost).

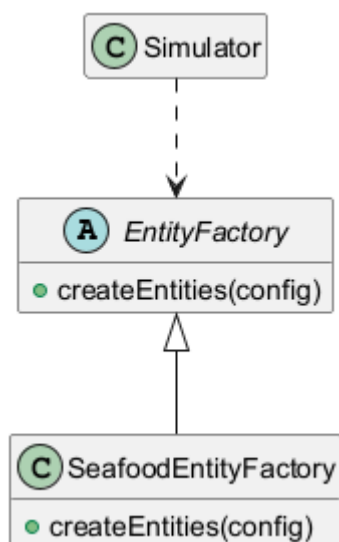
- **FactoryConsumptionReport** - detailně analyzuje produkci: který pracovník co vyrobil, podle jakého receptu a s jakou efektivitou (KPI). *Pro* manažery výroby a procesní inženýry. Pozor na recepty, musí obsahovat příslušné ve výlovu ryby, jinak nic nebude zpracováno - vše bude prodáno.
- **OutagesReport** - eviduje spolehlivost IoT zařízení, záznamy o jejich poruchách v důsledku opotřebení a historii provedených oprav. *Pro* oddělení údržby a technickou správu facility.
- **PartiesReport** - žebříček neaktivnějších subjektů v řetězci na základě počtu vygenerovaných událostí (vytížení uzlů). *Pro* Supply Chain Management k identifikaci klíčových partnerů.
- **TransactionReport** - sumarizuje veškeré obchodní transakce, pohyby peněz a celkový finanční obrát v rámci celého systému. *Pro* finanční oddělení a obchodní ředitele.

Implementace

Návrhové vzory (Patterns)

Poznámky k jednotlivým návrhovým vzorům.

1. Abstract Factory



Účel: Poskytuje rozhraní pro vytváření rodin souvisejících objektů bez specifikace jejich konkrétních tříd.

Použití v projektu: odděluje logiku načítání a instanciac doménového modelu od samotného běhu simulace.

1. Abstraktní továrna (EntityFactory):

definuje kontrakt metody `createEntities(Configuration config)`. Třída `Simulator` (klient) závisí pouze na této abstrakci.

2. Konkrétní továrna (SeafoodEntityFactory):

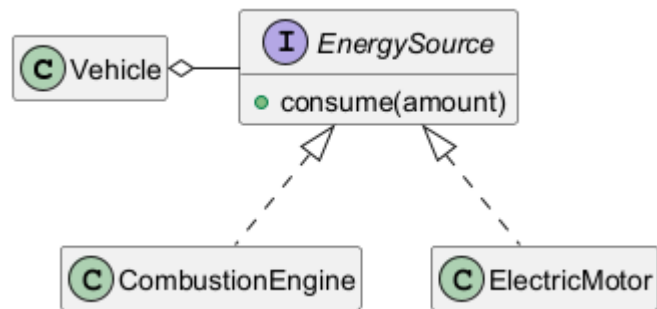
implementuje logiku pro doménu "Seafood". Zpracovává DTO objekty z konfigurace (`PartyConfig`, `DeviceConfig`) a převádí je na živé entity (`Fisher`, `ProcessingPlant`, `IndustrialFreezer`). Zapouzdřuje složitá pravidla tvorby (např. přiřazování strategií/rolí

`BusinessRole` jednotlivým stranám).

3. **Výhoda:** lze přidat jinou fabriku bez změny kódu simulatoru.

2. Strategy

Účel: Umožňuje dynamickou změnu chování objektu kompozicí místo dědičnosti. Navazuje na SOLID, a zejména 'Interface Segregation Principle' v návaznosti na 'Separation of Concerns' a 'Liskov's Substitution Principle'. Třídy by totiž neměly implementovat metody které nepoužívají, proto jejich funkcionality je jim zapůjčena zvenčí přes zástupnou proceduru. Výhodou je to, že funkcionality lze granulárně kombinovat.



Použití: nivelujeme "kombinatorickou explozi" dědičnosti, například:

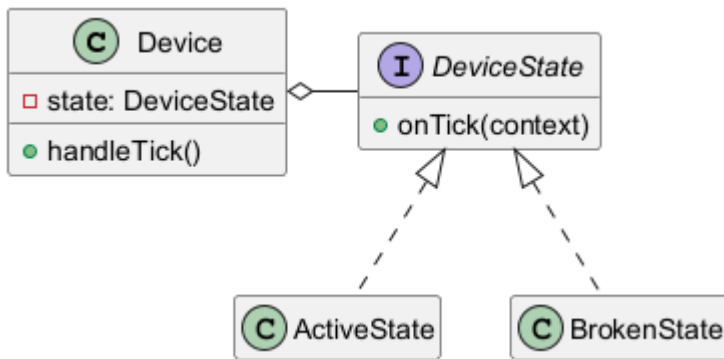
- `BusinessRole` určuje chování organizace (Rybář, Prodejce).
- `JobRole` určuje práci zaměstnance (Řidič, Manažer).
- `EnergySource` určuje spotřebu vozidla (Spalovací motor vs. Elektro).

Používá se přes přidání rolí do seznamu v instanci, např.:

```
public void addRole(BusinessRole role) {  
    this.roles.add(role);  
}
```

U rolí stran a pracovníků se používá **agregace** (list rolí lze měnit a jsou si navzájem nezávislé). Role zdrojů energie používají **kompozici** a injektují se do konstruktorů příslušných pohaněných vozů. (Pozn.: ne do konstruktéru).

3. State



Účel: Formalizuje koncept konečného automatu (Finite State Machine) v objektovém návrhu. Umožňuje objektu (Context) radikálně měnit své chování v závislosti na vnitřním stavu, což se navenek projevuje jako změna třídy. Vzor eliminuje neudržovatelné monolitické

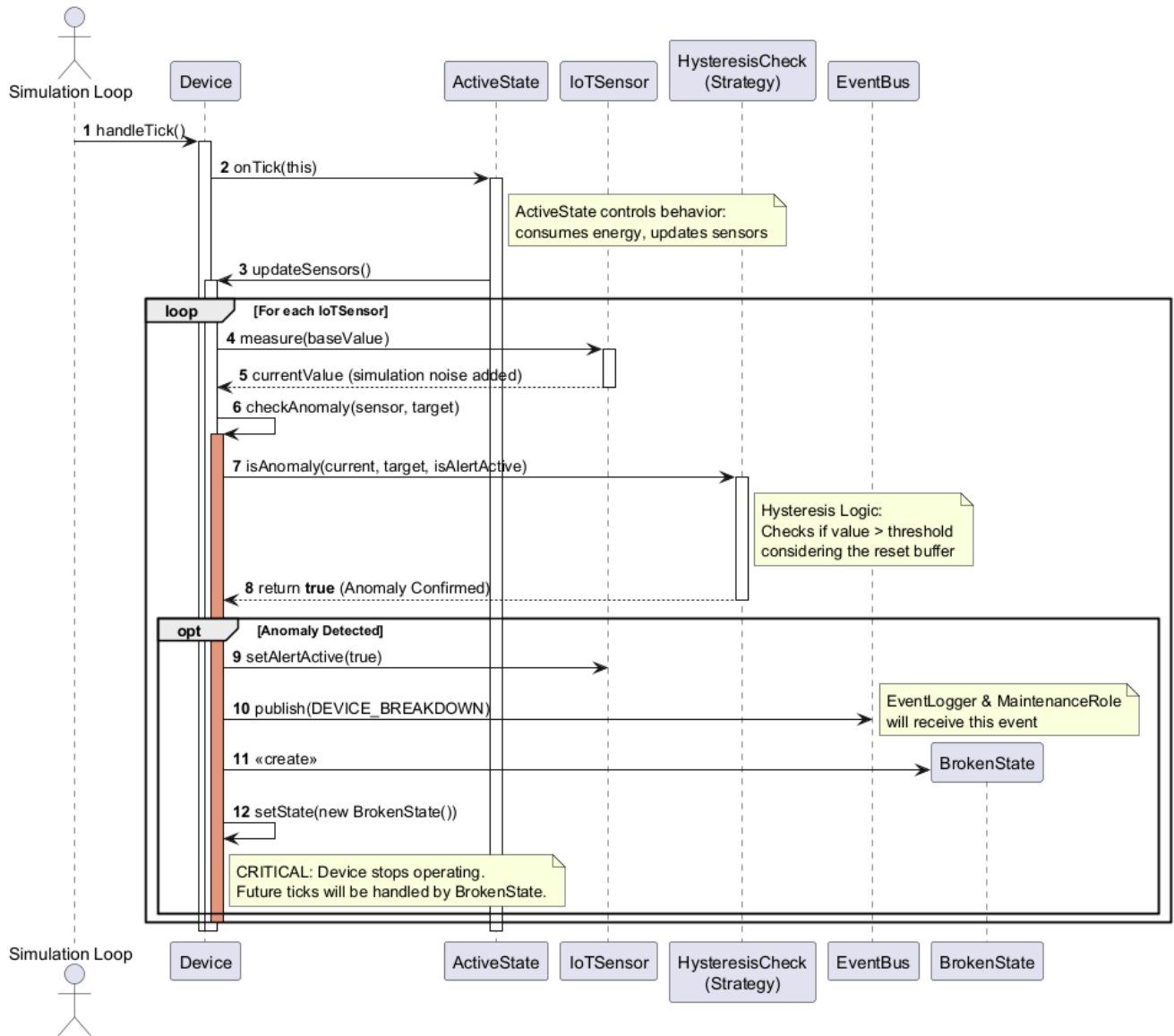
podmíněné větvení (`switch-case`) tím, že zapouzdřuje stavově závislou logiku do samostatných tříd. Tím zvyšuje soudržnost (cohesion) a dodržuje Open/Closed Principle – nové stavy lze přidávat bez modifikace kontextu.

Použití: v projektu tento vzor řídí dynamické chování entit, kde reakce na události (tik simulace, změna teploty) závisí na aktuální fázi životního cyklu:

1. **Device Lifecycle:** třída `Device` tvoří kontext, který deleguje volání na právě aktivní instanci `DeviceState` (`Active`). V tomto stavu stroj spotřebovává energii, opotřebovává se, rovněž se v něm aktivně monitorují parametry stroje, a při překročení prahových hodnot stav sám iniciuje přechod do jiného stavu (`Broken`), `Repairing` (resetuje se opotřebení). Vrací `DeviceState`.
2. **Item Lifecycle:** `Item` mění chování degradace kvality podle stavů `Caught`, `Stored`, `Processed`, `Sold`. Vrací `void`, mění navazuje na další stav uvnitř třídy stavu, dle GoF.

Zde je příklad sekvenčního diagramu stavů strojů:

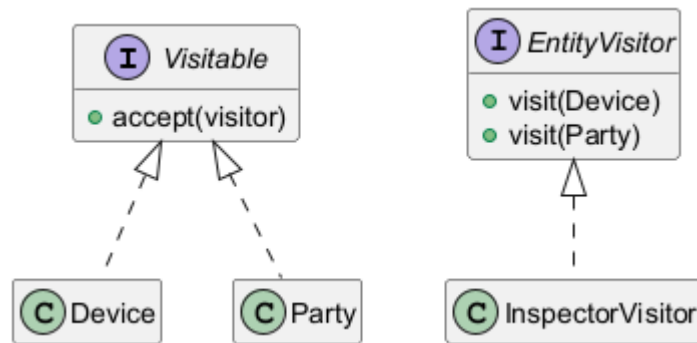
Device Anomaly Detection & State Transition Sequence



4. Visitor

Účel: Umožňuje přidávat nové operace nad objektovou strukturou bez nutnosti měnit třídy elementů, na kterých operace pracují. Umožňuje realizaci tzv. **Double Dispatch** v Javě.

Odděluje algoritmus (např. generování reportu) od datové struktury (Device, Party, Employee) a její změny.



Použití: v projektu se používá k auditu, reportování a injektování závislostí. Místo různých metod typu `printReport()`, `calculateTotalEnergy()` nebo `setDependencies()` v každé odpovídající třídě, třídy implementují pouze jednoduchou metodu `accept()`.

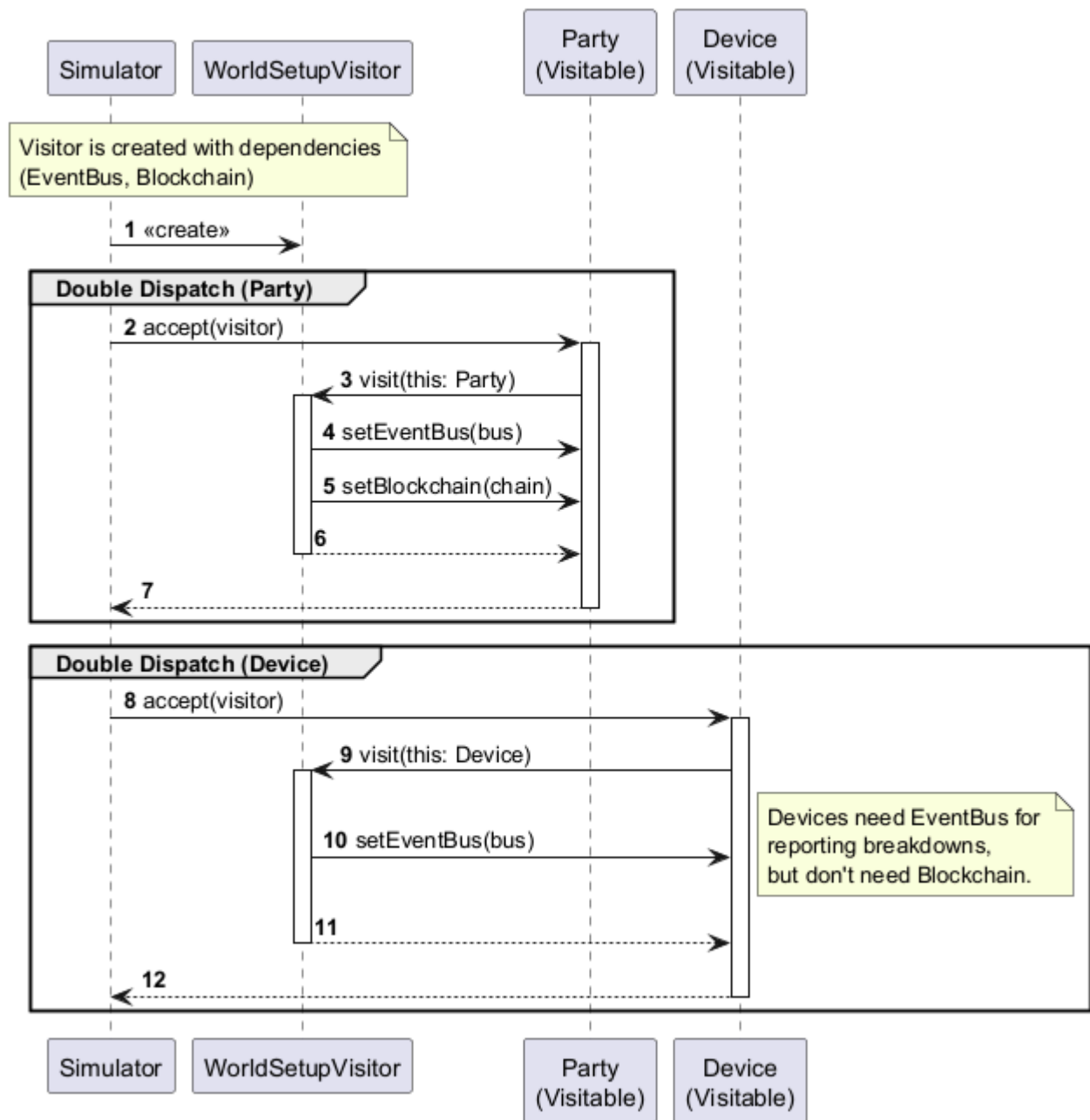
Konkrétní návštěvníci v systému:

- **WorldSetupVisitor:** prochází vygenerované entity a injektuje do nich závislosti (např. `EventBus`, `Blockchain`), které nebyly dostupné v době vytvoření v továrně.
- **ConsumptionVisitor:** agreguje `totalEnergyConsumed` ze všech zařízení napříč firmami a vypočítává celkovou energetickou náročnost simulace.
- **InspectorVisitor:** simuluje externí inspekci kvality. Obsahuje logiku kontroly pro různé typy entit (např. zda má `Device` v pořádku revize nebo `Employee` platné certifikáty), aniž by tato logika "znečišťovala" doménové modely.

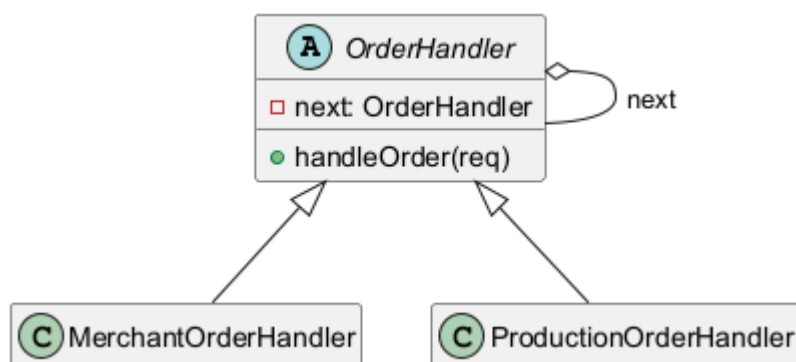
Role Inspektora (`InspectionRole`): třída `InspectorVisitor` obsahuje logiku kontroly pro různé typy entit (`Device`, `Party`, `Employee`), aniž by tato logika znečišťovala modely.

Zde je příklad sekvenčního diagramu pro injektování závislostí:

World Setup via Visitor (Double Dispatch)



5. Chain of Responsibility



Účel: Zamezuje pevné vazbě mezi odesílatelem požadavku a jeho příjemcem tím, že dává více objektům šanci požadavek vyřídit. Požadavek prochází řetězcem objektů, dokud jej jeden z nich nezpracuje.

Použití: Zpracování poptávky (`OrderHandler`). Pokud obchodník nemá zboží na skladě (`MerchantOrderHandler`), předá požadavek svému dodavateli nebo výrobě (`ProductionOrderHandler`). Implementuje "Pull" logiku řetězce.

Konkrétní články řetězce:

1. **StockHandler:** Nejprve zkontroluje lokální sklad (`Inventory`). Pokud je zboží dostupné, objednávku vyřídí okamžitě.
2. **DistributorOrderHandler:** Pokud zboží není na skladě, požadavek se předá distributorovi, který hledá dodavatele v jiných regionech.
3. **FisherOrderHandler:** Konec řetězce. Pokud zboží nikdo nemá, požadavek doputuje k výrobcí (rybáři), což vyvolá událost `DEMAND_CREATED` (nová poptávka po lovu).

Architektonické rozhodnutí: Pro implementaci byla zvolena **abstraktní třída** (`abstract class OrderHandler`) místo rozhraní. To umožňuje definovat logiku předávání (`nextHandler`, `passToNext`) na jednom místě a dodržet princip **DRY** (Don't Repeat Yourself), čímž se eliminuje duplicitní kód v jednotlivých handlerech.

6. Builder

Účel: Odděluje konstrukci složitého objektu od jeho reprezentace. Může být realizován klasický (s tzv. Directorem), který získává objednávku a vrací kompletní immutable objekt, anebo poskytovat tzv. fluent rozhraní - možnost přidávat vlastnosti postupně uživatelem, ať už idempotentně, či opakovaně. Samotná instanciací probíhá v závěrečném kroku.

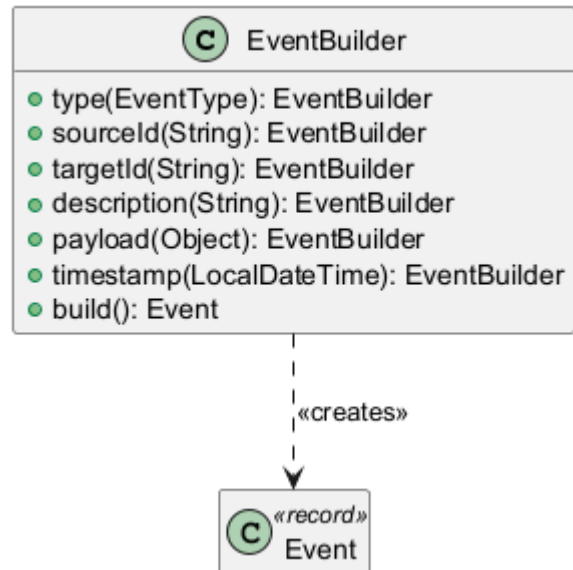
Použití: V našem případě třída

`EventBuilder` slouží k vytváření neměnných (immutable) objektů `Event`.

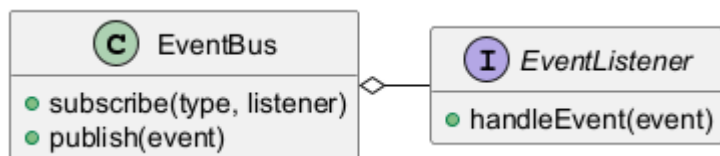
Zajišťuje čitelnost kódu pomocí plynulého rozhraní (fluent interface) a kaskádovému volání metod (method chaining); validace použitých a povinných polí je prováděná na konci, před vytvořením instance, voláním metody `build()` se vrací neměnitelný objekt typu `Event` (neměnitelný, protože je `record`).

V projektu je rovněž využit automatizovaný Builder pomocí anotace `@Builder` z knihovny Lombok u třídy `Transaction`, pro šetření místa.

Poznámka: Flexibilita Builderu umožňuje vložení libovolného objektu. Typová bezpečnost je proto vynucena až na straně příjemce (runtime check), kde je nutné ošetřit nekompatibilní typy dat (prevence `ClassCastException`). V praxi to znamená, že je potřeba provádět kontrolu 'Object' při přetypování (if `instanceof`, zamezení chybám, graceful failure) při zpracování události (event processing).



7. Observer (Publish-Subscribe)



Účel: Definuje závislost typu „jeden k mnoha“ mezi objekty tak, že změna stavu jednoho objektu vyvolá automatickou notifikaci a aktualizaci všech

závislých objektů.

Použití: třída `EventBus` funguje jako centrální uzel. Zařízení (`Device`) a lidé (`Employee`) publikují události (např. porucha, dokončení směny), na které reagují ostatní komponenty (např. `EventLogger`, `Inspector`).

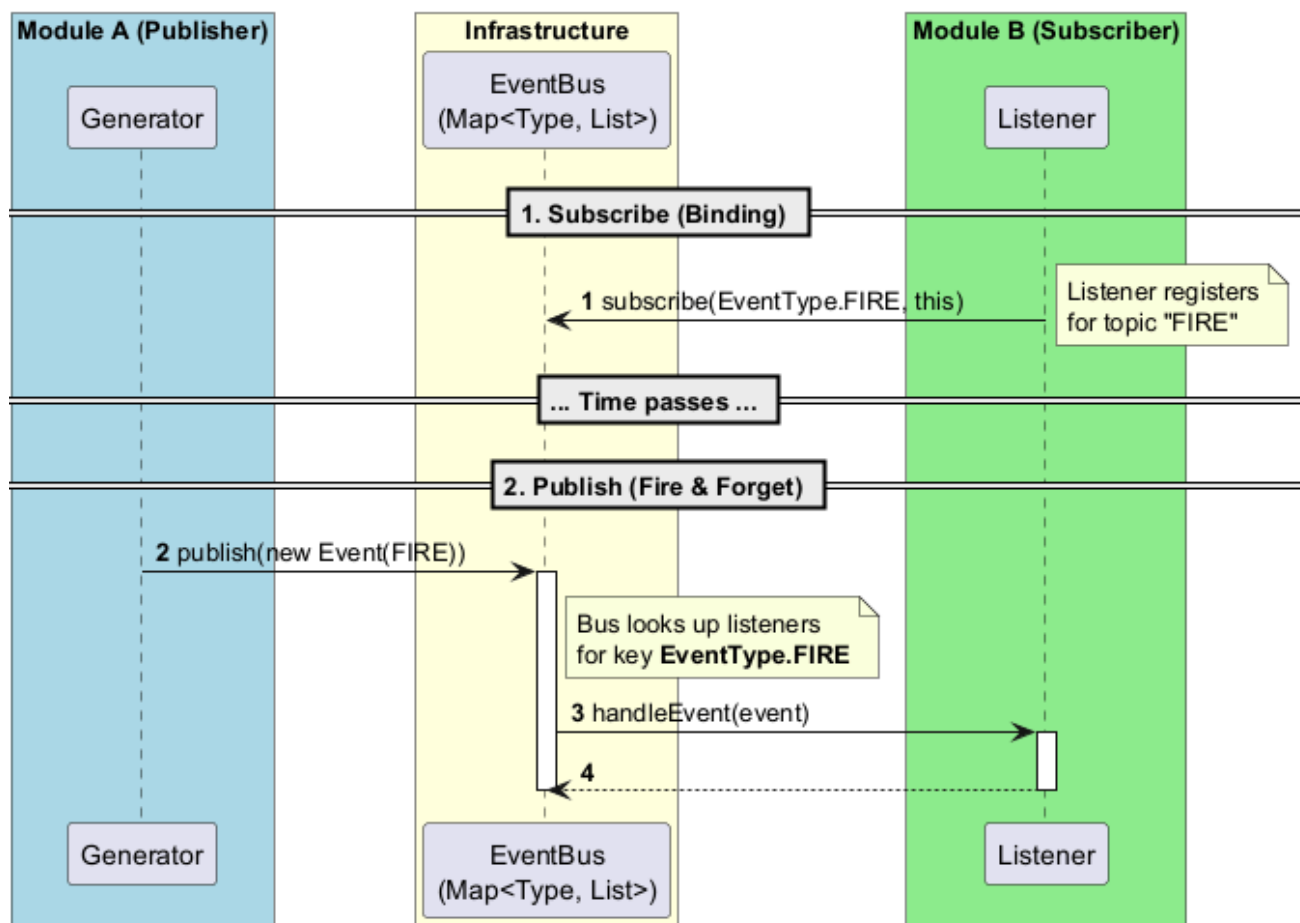
Implementace:

1. **EventBus (Broker):** Centrální kanál, který spravuje registraci posluchačů (`Map<EventType, List<EventListener>>`). Na rozdíl od klasického vzoru *Mediator*, `EventBus` neobsahuje business logiku interakce, pouze směruje zprávy podle typu události.
2. **Subscriber (Observer):** Rozhraní `EventListener` s metodou `handleEvent(Event e)`. Implementují jej třídy, které chtějí reagovat na změny (např. `EventLogger`, `MaintenanceRole`, `ManagementRole`).
3. **Publisher (Subject):** Entity jako `Device` nebo `Employee` publikují změny stavu voláním `eventBus.publish()`.

Příklad z kódu: Když `Device` detekuje opotřebení (`ActiveState`), publikuje událost `DEVICE_BREAKDOWN`.

- `EventLogger` (Observer 1) událost zapíše do souboru.
- `MaintenanceRole` (Observer 2) událost zachytí a přidá zařízení do fronty oprav.
- Samotné zařízení přitom neví, že existuje nějaký Logger nebo Údržbář.

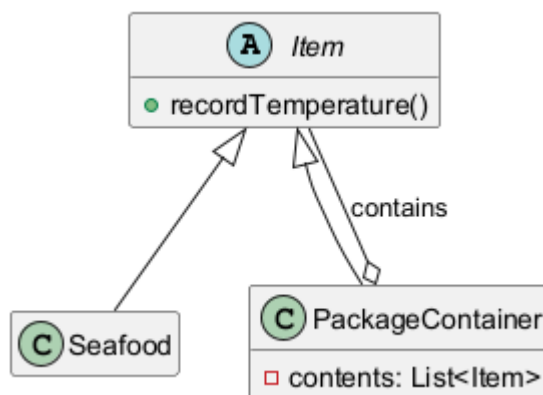
Pattern: Event Bus (Pub-Sub)



8. Composite

Účel: Umožňuje klientům přistupovat k jednotlivým objektům a jejich kompozicím (skupinám) jednotným způsobem. Vytváří stromovou strukturu „část-celek“ (Part-Whole).

Použití: V projektu tento vzor řeší hierarchii balení a skladování produktů (`Item`). Skladník nebo přepravce (klientský kód) nemusí rozlišovat, zda manipuluje s jednou rybou, krabicí plnou ryb, nebo paletou plnou krabic. Třída `PackageContainer` (krabice, paleta) může obsahovat jiné `Item` (ryby nebo další krabice). Metody jako `recordTemperature()` se rekurzivně volají na celý obsah.

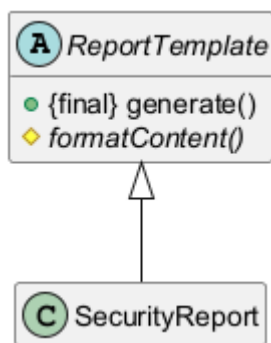


Implementace:

1. **Component (Item):** Abstraktní třída definující společné rozhraní, například `recordTemperature(double temp)` nebo `getTotalWeight()`.
2. **Leaf (Seafood, FinishedProduct):** Konkrétní produkty (listy stromu), které nemají podřízené prvky. Metody vracejí konkrétní hodnoty (např. vlastní váhu).
3. **Composite (PackageContainer):** Představuje kontejnery (krabice, palety). Obsahuje seznam `List<Item> contents`.
 - **Rekurzivní logika:** Metody kontejneru delegují práci na své potomky.
 - Příklad `getTotalWeight()` : Vrací součet vlastní váhy obalu (tára) + součet vah všech vnořených položek.
 - Příklad `recordTemperature()` : Zaznamená teplotu pro sebe a následně ji propaguje na všechny položky uvnitř (pokud zmrzne paleta, zmrznou i ryby uvnitř).

Tento přístup radikálně zjednodušuje kód pro manipulaci se zásobami (Party), protože operace nad celým stromem objektů se volají stejně jednoduše jako nad jediným objektem.

9. Template Method



Účel: Definuje kostru algoritmu v operaci a některé kroky přenechává podtřídám. Umožňuje podtřídám redefinovat určité kroky algoritmu bez změny jeho struktury.

Použití: V projektu zajišťuje jednotný proces generování výstupních reportů. Garantuje, že každý report bude mít správnou hlavičku, časové razítko a bude korektně uložen na disk, přičemž samotný obsah reportu se liší dle jeho typu.

Generování reportů (ReportTemplate). Metoda `generate()` definuje pevnou strukturu (hlavička → obsah → patička → uložení), zatímco podtřídy (FoodChainReport, SecurityReport) implementují pouze specifické formátování obsahu.

Implementace:

1. **Abstraktní třída (ReportTemplate):**
 - Obsahuje metodu `generate(List<Event> history)`, která je označena klíčovým slovem `final`. To je kritické, protože to zabraňuje podtřídám změnit sled kroků (Hlavička -> Obsah -> Patička -> Uložení).
 - Definuje abstraktní (hook) metodu `formatContent()`, kterou musí potomci implementovat.
2. **Konkrétní třídy (SecurityReport, FoodChainReport, TransactionReport):**
 - Implementují pouze logiku formátování specifických dat.

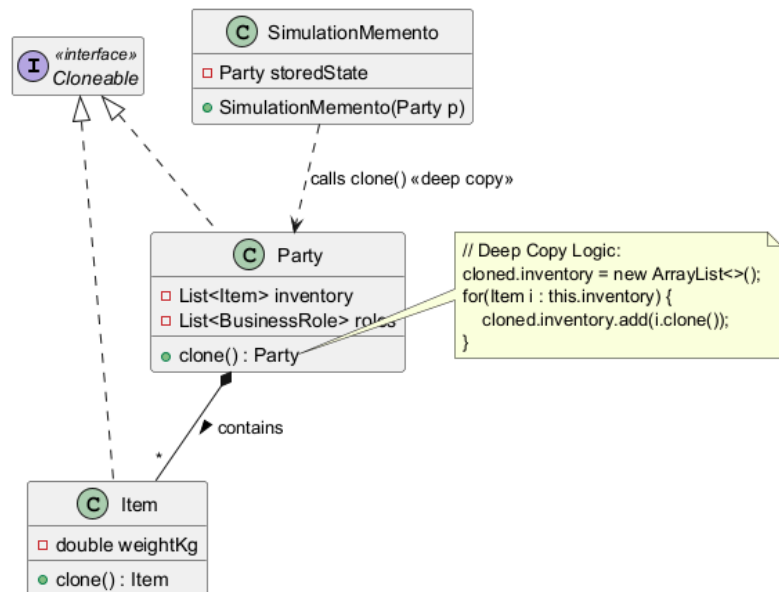
- Příklad: `SecurityReport` filtruje z historie pouze události `BLOCKCHAIN_TAMPERING_DETECTED`, zatímco `FoodChainReport` vypisuje pohyby zboží.

Vzor dodržuje "Hollywood Principle" (Don't call us, we'll call you) – rodičovská třída řídí tok a volá metody potomků v přesně určený okamžik.

10. Prototyp

Účel: Umožňuje vytváření kopií objektů bez závislosti na jejich konkrétních třídách. V kontextu projektu je klíčový pro **Deep Copy** (hlubokou kopii) složitých objektů.

Použití: Vzor je nezbytný pro správnou funkčnost vzoru **Memento**. Abychom mohli uložit historický stav firmy (`Party`), musíme vytvořit její přesnou kopii. Pokud bychom použili pouze referenci (Shallow Copy), změny v aktuálním čase by přepsaly i uloženou historii, což by znehodnotilo snapshoty.

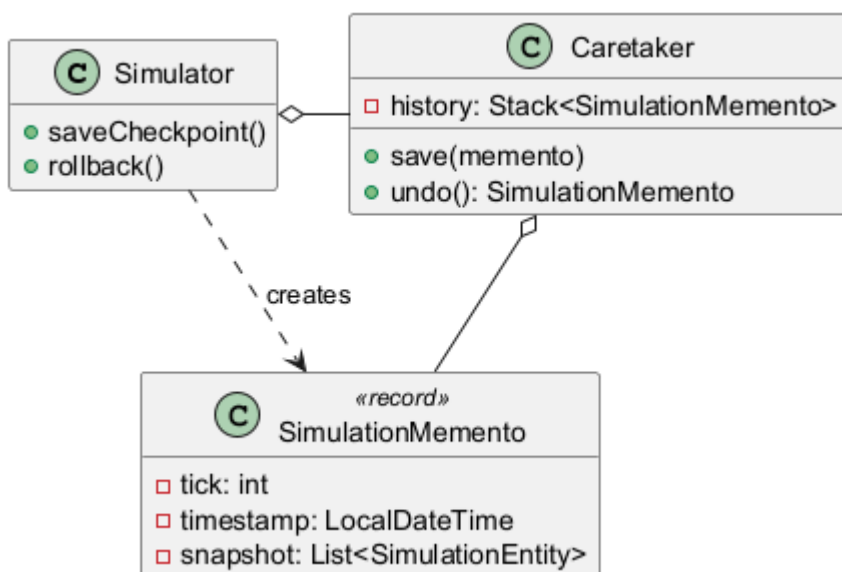


Implementace:

- Rozhraní `Cloneable`:** Třídy `Party` a `Item` implementují toto standardní Java rozhraní.
- Metoda `clone()`:**
 - `Party`: Nevytváří jen novou instanci `Party`, ale prochází svůj `inventory` (sklad) a pro každou položku volá její vlastní metodu `clone()`. Tím vzniká zcela nezávislý seznam zboží.
 - `Item`: Vytváří kopii sebe sama, aby se oddělil stav (např. váha, kvalita) v historii od současnosti.

Díky tomuto vzoru může `Caretaker` (správce historie) kdykoliv požádat: "Vytvoř mi otisk aktuálního stavu" a objekt `Party` se sám bezpečně zduplikuje.

11. Memento



Účel: Umožňuje zachytit a externalizovat vnitřní stav objektu, aby mohl být později obnoven, aniž by došlo k porušení zapouzdření.

Použití: V projektu slouží primárně k mechanismu

Emergency Rollback. Pokud auditní systém (IoT sensory nebo

Blockchain) detekuje narušení integrity - například **Double Spending** (pokus o dvojí prodej stejné šarže) - simulace se automaticky vrátí k poslednímu validnímu kontrolnímu bodu (checkpointu).

Implementace:

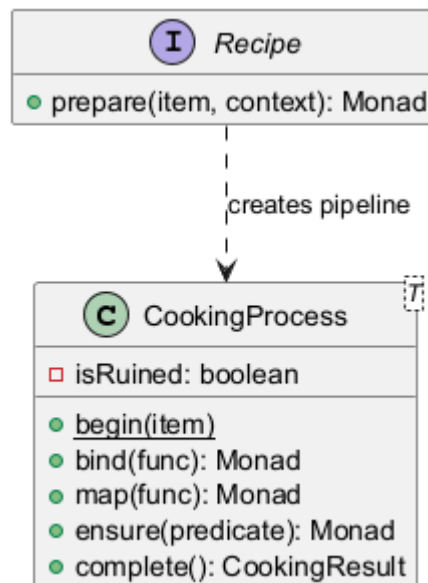
1. **Originator (SimulationController):** Objekt, který řídí běh světa. Pravidelně (např. každých 24 ticků) vytváří snímek stavu. Pro zajištění věrnosti minulosti vytváří **hlubokou kopii (Deep Copy)** všech entit a jejich inventářů pomocí vzoru **Prototype**.
2. **Memento (SimulationMemento):** Imutabilní záznam (Java Record), který uchovává zmrazený seznam naklonovaných entit, aktuální tick a časové razítko.
3. **Caretaker (HistoryManager):** Spravuje historii mement na zásobníku (Stack). Zajišťuje, že se můžeme vrátit o několik kroků zpět, pokud je detekováno více po sobě jdoucích incidentů.

Klíčový přínos pro audit: Zatímco stav entit a čas v simulátoru se po obnově vrátí do minulosti, **EventLogger** a **Blockchain** zůstávají netknuté. To znamená, že v konečném reportu je vidět jak pokus o podvod (**MaliciousActor**), tak i následná úspěšná obnova systému, což prokazuje vysokou odolnost celého dodavatelského řetězce.

12. Monad (Functional Pattern)

Účel: Řetězení operací s automatickým zpracováním vedlejších efektů, stavů a chyb (tzv. **Railway Oriented Programming**). Umožňuje bezpečně skládat funkce bez nutnosti manuálního ošetřování chyb v každém kroku.

Použití: Třída `CookingProcess<T>`. Tento vzor se používá v receptech (`Recipe`) k definici výrobního postupu jako série transformací. Pokud jakýkoliv krok selže (např. chybí surovina, porucha stroje), zbytek řetězce se přeskočí. Tím se eliminuje tzv. "Pyramid of Doom" (hluboce vnořené podmínky `if-else`). Na rozdíl od Builderu, kde pořadí metod není důležité - jedná se o strukturu jednoho objektu, zde Monada musí dodržovat pořadí, výstup z jedné funkce je vstupem druhé, a na pořadí záleží. Dále, a to je ještě důležitější, - nevytvoří například sushi bez ryby. Takže nám třída `CookingProcess<T>` dovoluje flexibilně navrhovat recepty do kuchyně.



Implementace:

1. **Context Wrapper:** `CookingProcess` zapouzdřuje surovinu a aktuální stav operace.
2. **Bind / FlatMap:** Metoda (např. `thenProcess()`), která přijímá funkci definující další krok zpracování.
3. **Failure Handling:** Interní logika, která při detekci chyby (Exception nebo prázdný stav) okamžitě vrací "Error context", čímž efektivně zastavuje další výpočty v daném řetězci.

Teoretické myšlenky o monadě :)

Obecně, *monáda* je návrhový vzor, který využívá pojmy z teorie množin a teorie kategorií v matematice, přičemž vychází zejména z konceptu *monoidu*. Monoid je množina, která je uzavřená vzhledem k binární asociativní operaci a obsahuje neutrální prvek. To znamená:

- *Uzavřenost* — operace nad prvky množiny vrací vždy prvek z téže množiny. V kontextu teorie kategorií to umožňuje definovat tzv. *endofunctor* — zobrazení typu na typ uvnitř jedné kategorie. Kategorie je tvořena objekty a morfismy (vztahy) mezi nimi; endofunctor tuto strukturu zachovává.

- *Asociativita* — prvky se spojují po dvou, přičemž na uzávorkování (pořadí provádění operací) nezáleží, záleží však na pořadí prvků: $(AB)C = A(BC) = ABC$. Platí však, že $AB \neq BA$ (komutativita není vyžadována, což je u sekvenčních procesů klíčové; například u vzoru Builder by na pořadí metod záležet nemuselo, u monády ano).
- *Neutrální prvek* — prvek identity, který při operaci s jiným prvkem nemění jeho hodnotu. Například prázdný řetězec `""` pro spojování řetězců, `0` pro sčítání nebo `1` pro násobení.

Monáda zajišťuje chování výpočetních operací tak, že jejich kompozice tvoří monoid. Zatímco klasický monoid je trojice (*Množina prvků, Asociativní operace, Neutrální prvek*), monádu lze chápat jako trojici:

1. Funkce vracející *hodnotu* zabalenou v kontejneru/kontextu (uzavřená množina Kleisliho šipek/funkcí).
2. Binární operace spojování funkcí `bind` (v Javě `map` pro čistý prvek nebo `flatMap` pro prvek v kontejneru),
3. Neutrální operace `return` (v Javě např. `of`), která vrací výsledek zabalený do kontejneru/kontextu.

K tomu, aby byla operace skutečně neutrální, musí platit zákony identity:

1. *Levá identita* — monáda (jednotka) aplikovaná na funkci vrací stejný výsledek jako samotná funkce; je neutrální.
 - programově: `Return(x).bind(f) == f(x)`
 - matematický: `return x >=> f ≡ f x`,
2. *Pravá identita* — pokud na existující monádu aplikujeme funkci `return` (zabalení do monády), výsledek se nezmění.
 - programově: `m.bind(Return()) == m`
 - matematický: `m >=> return ≡ m`,

Toho se docílí správnou aplikací metody `bind` (`flatMap`).

V podstatě se jedná o sekvenční „pipeline“, kde každá funkce kromě výpočtu nese i vedlejší efekt (kontext). Při spojování těchto funkcí monáda definuje, jak s tímto kontextem zacházet, aby byla zachována asociativita. Operace `return` pak představuje neutrální funkci/prvek. V rámci monády se obvykle pracuje s tzv. čistými funkcemi (pure functions), zatímco monáda samotná zapouzdřuje a řídí vedlejší efekty. Pokud funkce není čistá, je dobře ji na čistou funkci převést, tj. zajistit aby funkce vracela vedlejší efekt jako součást svého výpočtu. Monáda pak řídí, jak se tyto efekty propagují a kombinují.

Příklad:

```
// Funkce s vedlejším efektem (vyhození vyjímky)
Integer divide(int a, int b) { return a / b; } // Hrozí ArithmeticException (vedlejší
efekt havárie)
```

```
// Čistá funkce (Pure) - vrací Monádu (Optional)
Optional<Integer> divide(int a, int b) {
    if (b == 0) return Optional.empty(); // Chyba je nyní hodnotou (kontextem)
    return Optional.of(a / b);
}

// Použití v pipeline (bind/flatMap):
Optional.of(10)
    .flatMap(x -> divide(x, 2)) // OK: Optional[5]
    .flatMap(x -> divide(x, 0)); // OK: Optional.empty (pipeline se bezpečně zastaví)
```

Monáda využívá generický typ (`<T>`), aby vyhovovala definici Endofunktoru — musí být schopna zobrazovat typ na typ uvnitř stejné kategorie (v našem případě kategorie typů v Javě). Konkrétně u nás `CookingProcess<T extends Item>` mapuje různé typy `T` (odvozené od `Item`) na typ `CookingProcess<T>`. Teoreticky bychom mohli použít i širší definici `CookingProcess<T>` (pro libovolný objekt), ale omezení na `Item` nám poskytuje typovou bezpečnost v rámci domény projektu.

Poznámka: „Monáda je pouze monoid v kategorii endofunktorů“. Na základě osobního pozorování zdrojů na webu, v ruské programátorské praxi se namísto pojmu monáda někdy používá stejný pojem jako v matematice — monoid.

Slovník pojmů a slové zkratky

- SCM — Supply chain management.
- *Bod rozpojení* (anglicky *decoupling point* nebo *customer order decoupling point*) — klíčový pojem v logistice a řízení výroby. Označuje místo v logistickém řetězci, kde materiálový tok začne být určen pro konkrétního zákazníka a jeho objednávku. Jinými slovy — místo, kam až dosahuje poptávka konkrétního zákazníka, před toto místo se vyrábí "na sklad". Používá se při řízení zásob. Zákazníci jsou různé a mohou se nacházet podél celého LŘ, a tak i jim příslušné body rozpojení.
- Monad — is a monoid in the category of endofunctors [12].

Reference

1. Nástroj pro tvorby UML, UMLET, URL: <https://umlet.com/>
2. Nástroj pro psaní v markdown a export do PDF: Typora, v. 1.9, URL: <https://typora.io/releases/stable.html>
3. Monada:
 1. No Nonsense Monad & Functor - The foundation of Functional Programming by César Tron-Lozai, URL: <https://www.youtube.com/watch?v=e6tWJD5q8uw>
 2. Monad is actually easy. (Conquering the Final Boss of Functional Programming), Coding with Yalco, URL: <https://www.youtube.com/watch?v=8hYUthfmSRM>
 3. Что значит моноид в программировании? S0ER, URL: <https://www.youtube.com/watch?v=Da-avtMWlf8>
 4. What is a monad? (Design Pattern), A Byte of Code, URL: <https://www.youtube.com/watch?v=VgA4wCaxp-Q>
 5. What is a Monad? — Math vs Computer Science, Sheafification of G, URL: https://www.youtube.com/watch?v=roP_HC7tiXw
 6. What is a Comonad? — Comath and Mputer Science, Sheafification of G, URL: <https://www.youtube.com/watch?v=T8pjE2JMv-c>
 7. Monads in Java, URL: <https://www.baeldung.com/java-monads>
 8. Functor and monad examples in plain Java, Tomasz Nurkiewicz, URL: <https://nurkiewicz.com/2016/06/functor-and-monad-examples-in-plain-java.html>
 9. Моноиды и их приложения: моноидальные вычисления в деревьях, URL: <https://habr.com/ru/articles/112394/>
 10. Моноид без слез, URL: <https://sqrtrt.pro/monoids-without-tears-ru>

11. Monads, Bartosz Milewski's Programming Cafe, URL: <https://bartoszmilewski.com/category/monads/>
12. A Brief, Incomplete, and Mostly Wrong History of Programming Languages, One Div Zero, URL: <https://james-iry.blogspot.com/2009/05/brief-incomplete-and-mostly-wrong.html>